

MLDS Exploratory Data Analysis & Visualisation

Week 9 - Data Reduction

Spring Term, 2025

- Introduction
 - The MNIST Dataset
- Principal Component Analysis
 - Benefits and Limitations
- Multidimensional Scaling
 - Benefits and Limitations
- t -distributed Stochastic Neighbour Embedding
 - Benefits and Limitations
- References

Introduction

High dimensional data is ubiquitous in today's Big Data era. In many applications, data scientists face the challenge of working with data whose dimension can run to the order of thousands or even higher. For example, in gene sequencing data, the dimension is given by the number of genotypes, which is often in the order of thousands; for datasets of images, the dimensionality is given by the number of pixels in each image, which can range from thousands to millions.

From an EDA point of view, there are a number of questions we might ask ourselves, When presented with a high-dimensional dataset. We might reasonably wonder if all of the provided dimensions are equally useful; can we distill the data into a smaller (i.e. lower-dimensional) dataset, and still retain a sufficient portion of the information contained within? We might also be inclined to try to visualise the dataset as a whole, representing all observations on the same graphic. This would be difficult to do so in an interpretable way for a very high-dimensional dataset, but we can make this task easier by projecting onto a lower-dimensional (visualisable) space. If our eventual modelling goal is to produce a classifier, then another useful part of exploratory analysis would be to establish whether observations of the same class share similar projections onto a lower-dimensional (visualisable) geometric space.

Each of these exploratory questions involve reducing the dimensionality of the dataset. Note that, although our motivation is the *exploratory* analysis of data (i.e. without considering the application of specific machine learning models), many machine learning algorithms struggle to work effectively on high-dimensional datasets, and so dimension reduction techniques should therefore be seen as a class of auxiliary tools for machine learning in general, and with much broader applicability in data science than as part of exploratory data analysis.

In this week's notebook, we will go through some popular methods that enable us to reduce the dimensionality of data whilst retaining most of the information. In Applicable Maths, we saw that Principal Component Analysis (PCA) can be used to perform exactly this task. Below, we remind ourselves of the basics of PCA, and demonstrate its use on our MNIST data subset. We will then consider two further data reduction techniques: multidimensional scaling (MDS) and t -distributed stochastic neighbour embedding (t-SNE). For each method, we will briefly recap how it works, discuss its benefits and limitations, and demonstrate its implementation with a dataset as a running example.

The MNIST Dataset

We will use the MNIST dataset (<http://yann.lecun.com/exdb/mnist/>). This is a collection of digitised images of handwritten numeric digits (0 to 9). The full dataset contains 70,000 images, and is often used to demonstrate pattern recognition algorithms. Each image in the dataset comprises $28 \times 28 = 784$ pixels and the (grayscale) colour of each pixel can be represented by an integer value from 0 to 255 (where 0 and 255 indicate white and black, resp.). We can thus treat this as a multivariate dataset, containing 70,000 observations, with a dimensionality of 784.

The MNIST dataset is widely available online, in various formats, though it is very large. We will instead work with a subset of 1000 observations from the MNIST dataset; this will suffice for the demonstrations and exercises included in this notebook. You are, of course, free to locate and download the entire dataset yourself, but note that some of the functions in this notebook will then place huge memory requirements on your machine and take a long time to finish.

The subset of 1000 MNIST observations is provided in CSV format alongside this markdown file. When you first load the data from this file, the resulting dataframe will contain the true digit that each observation depicts, in the first column (`digit`), followed by the grayscale pixel values in the remaining 784 columns. We perform a small amount of preprocessing of the data, before getting stuck into data reduction methods: we normalise the grayscale values in each column (apart from the first), such that values of 0 and 1 subsequently indicate white and black, respectively, and we define `mnist_x` to be the dataframe containing *only* the pixel values (i.e. ignoring the true digit label).

```
#mnist <- read.csv("./mnist_1000obs.csv")
mnist <- readr::read_csv("./mnist_1000obs.csv")
```

```
## Rows: 1000 Columns: 785
## — Column specification
##
## Delimiter: ","
## dbl (785): digit, pixel.1, pixel.2, pixel.3, pixel.4, pixel.5, pixel.6, pxe...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this messag
e.
```

```
preprocess <- function(mnist) {
  digit <- as.factor(mnist$digit)
  mnist <- mnist %>% select(-digit) # remove labels
  mnist <- mnist / 255 # normalise pixel values to [0, 1]
  mnist$digit <- digit # add back labels

  return(mnist)
}

dim(mnist)
```

```
## [1] 1000 785
```

```
mnist <- preprocess(mnist)
mnist_x <- mnist %>% select(-digit)
```

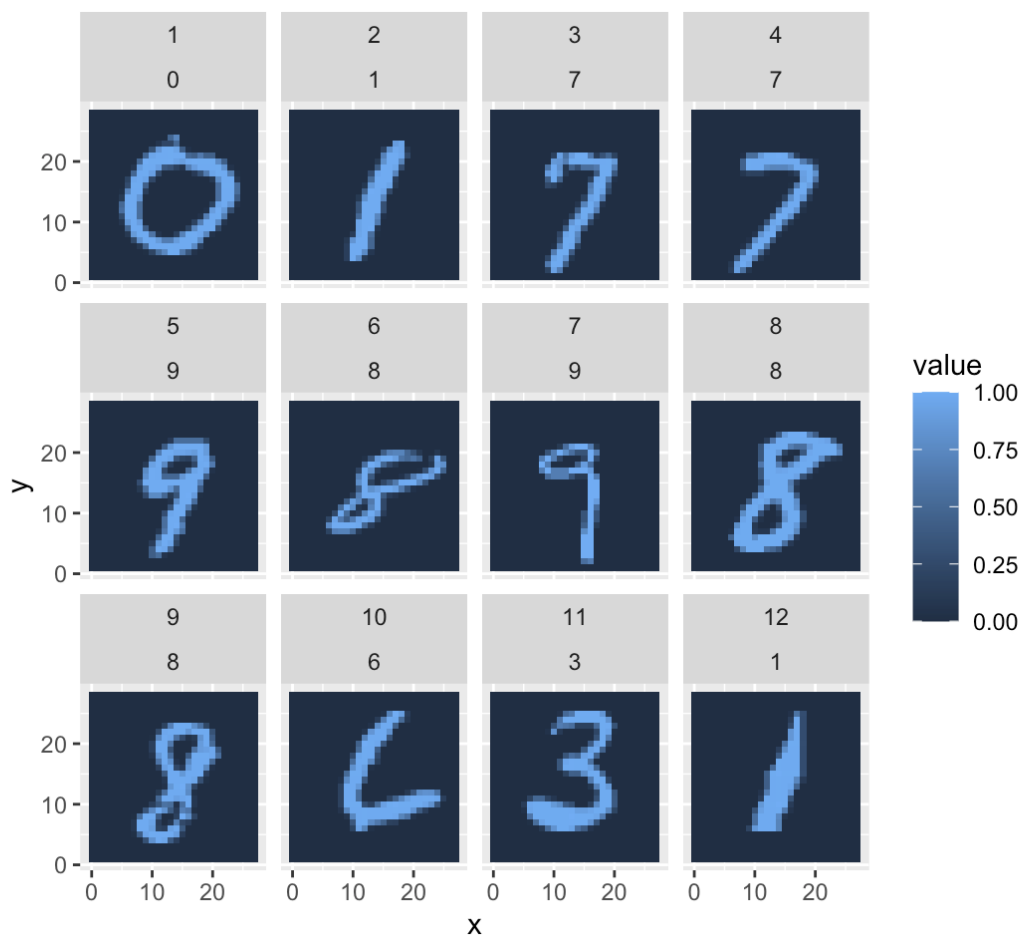
We will also define an useful auxiliary function for visualising a single observation (i.e. for graphically displaying an image of a handwritten digit).

```
visualise = function(df, labels, nrow, ncol, ...){ # function for graphically displaying a digit
  df <- bind_cols(df, digit=labels)

  pixels_gathered <- df %>%
    mutate(instance = row_number()) %>%
    gather(pixel, value, -digit, -instance) %>%
    tidyr::extract(pixel, "pixel", "(\\d+)", convert = TRUE) %>%
    mutate(pixel = pixel - 1,
           x = pixel %% 28,
           y = 28 - pixel %/% 28)

  ggplot(data = pixels_gathered, aes(x, y, fill = value)) +
    geom_tile() + coord_fixed() +
    facet_wrap(~ instance+digit, nrow=nrow, ncol=ncol)
}

# randomly sample 12 images to show
ind <- sample(1:nrow(mnist), 12)
visualise(mnist_x[ind, ], labels = mnist[ind, "digit"], nrow = 3, ncol = 4)
```



Principal Component Analysis

Principal Component Analysis (PCA), loosely put, finds the directions in which the data vary the most. These directions are called principal components, and the variance of the data along each of these directions is given by the corresponding principal value.

We have seen a detailed treatment of PCA in Chapter 3 of *Applicable Maths*; here, we briefly recap the key concepts. Let $X \in \mathbb{R}^{n \times p}$ be a matrix representing a dataset, where each row $x_i^T \in \mathbb{R}^{1 \times p}$ is an observation and each column corresponds to a covariate/feature value (X is also referred to as a design matrix). We assume the covariates are *centred* so that each column has mean 0,

i.e. $\bar{X}_{\cdot,j} := \frac{1}{n} \sum_{i=1}^n X_{ij} = 0$ for all $j = 1, \dots, p$.

The sample covariance matrix is given by $S = \frac{1}{n} X^T X = \frac{1}{n} \sum_i x_i x_i^T$ (as $x_i \in \mathbb{R}^p$ is a column vector). For $j = 1, \dots, p$, the j -th principal component, denoted by v_j , is defined to be the eigenvector associated with the j -th largest eigenvalue, λ_j , of S . The v_j and λ_j can be found by an iterative algorithm:

- The first principal component is the unit vector along which the variance is maximised:

$$v_1 = \arg \max_{\|u\|_2=1} \{u^T S u\},$$

and the associated principal value is $\lambda_1 = v_1^T S v_1$.

- The j -th principal component v_j , for $j = 2, \dots, p$ is the unit vector that is orthogonal to v_k for all $k < j$, and maximises the variance in the space orthogonal to the span of v_k for all $k < j$:

$$v_j = \arg \max_{\|u\|_2=1, u^T v_k=0, \forall k < j} \{u^T S u\}.$$

The associated principal value is $\lambda_j = v_j^T S v_j$.

In R, PCA can be performed using two alternative functions. `prcomp()` and `princomp()` (both of which are part of the stats package). You can uncomment the following lines of code to access the R documentation for these functions.

```
# ?prcomp
# ?princomp
```

`prcomp()` calculates the principal components through applying the singular value decomposition to the data matrix, whereas `princomp()` uses the eigendecomposition of the sample covariance matrix. In practice, the SVD approach is generally favoured over the use of the eigendecomposition, as it provides greater numerical stability; we therefore stick to using `prcomp()` in this notebook.

Note that `prcomp` has the options `center` and `scale.`. These control whether the input matrix X is transformed so that each column has zero mean and unit variance. The default is `center = TRUE` and `scale. = FALSE`. Note that, in PCA, it is *essential* for X to have zero mean (since otherwise S is no longer the sample covariance matrix). On the other hand, scaling to unit variance is strongly advisable and should be done whenever it makes sense. This is because PCA aims to analyse the variance, the magnitude of which strongly depends on the scale of the covariates. However, as we will see next, it does not always make sense to apply scaling to all data.

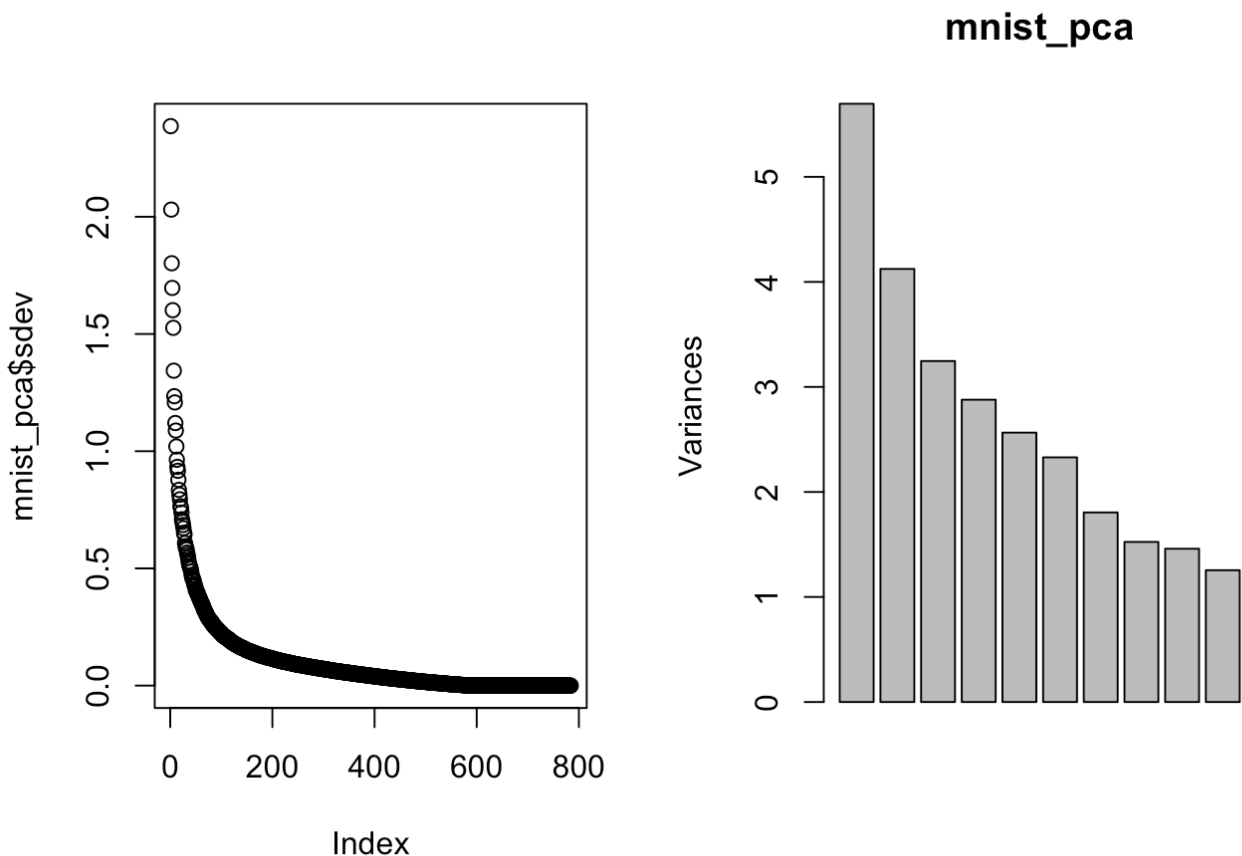
We apply PCA to `mnist` after excluding the columns of labels, `digit`. Note that we cannot set `scale. = TRUE` for this dataset, since some of the columns are all 0's, and so dividing by their standard deviation (which is 0) would cause an error.

```
mnist_pca <- prcomp(mnist_x) # this may take a few seconds
```

The standard deviations of the data along each of the principal components can be accessed through `pr$sdev` ; these are the square roots of the eigenvalues of S (and are equal to the square roots of the principal values).

```
par(mfrow=c(1,2))

plot(mnist_pca$sdev) # sqrt of the e.values
plot(mnist_pca)
```



There are many ways to choose how many principal components we should keep. One way is to look at the cumulative explained variance ratio as a function of components.

Exercises

1. Compute `cumsum_values` , the cumulative sum of the principal values using the function `cumsum` . The output should be a vector of length p .
2. Compute `sum_values` the sum of the principal values. The output should be a single float.
3. Compute the cumulative ratio of variance by dividing `cumsum_values` by `sum_values` .

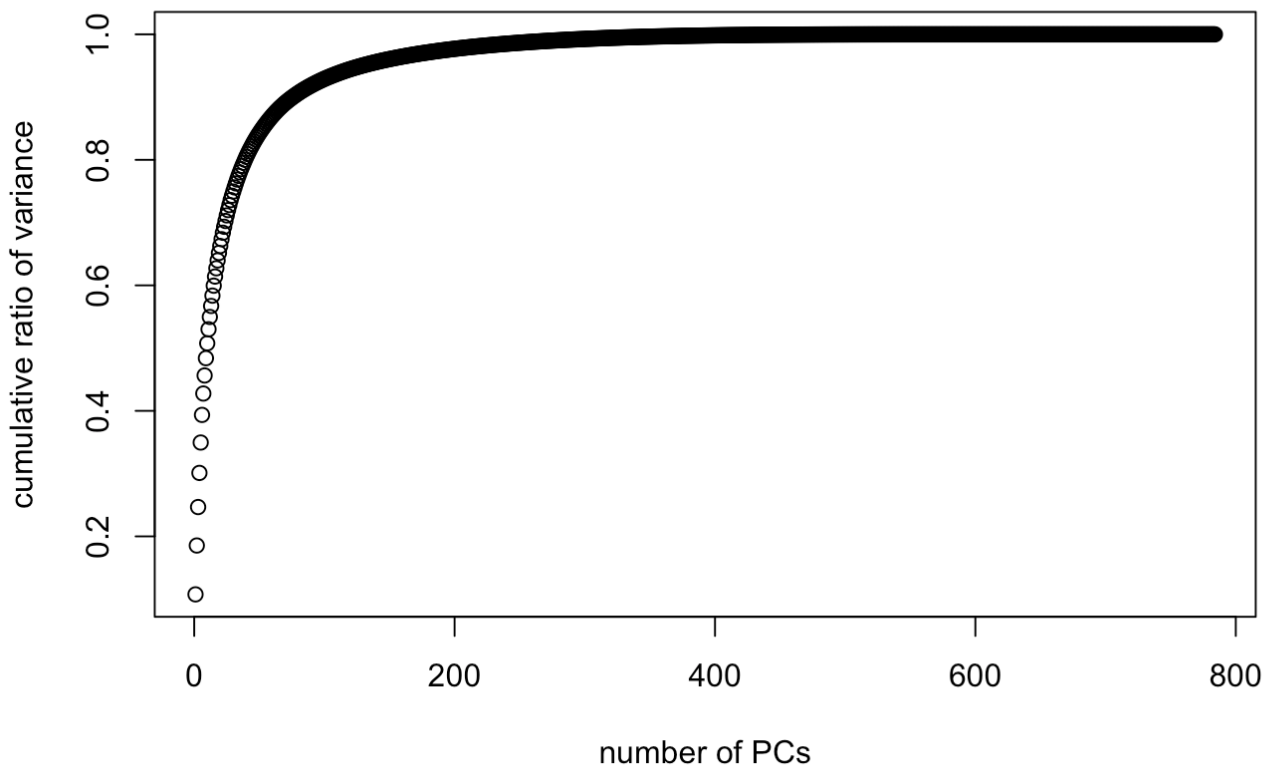
```
cumsum_values <- cumsum(mnist_pca$sdev^2) # Q1
print(length(cumsum_values)) # this should give 784
```

```
## [1] 784
```

```
sum_values <- sum(mnist_pca$sdev^2) # Q2
print(sum_values)
```

```
## [1] 52.95229
```

```
cumsum_prop <- cumsum_evalues / sum_evalues # Q3  
plot(cumsum_prop, xlab = "number of PCs", ylab = "cumulative ratio of variance")
```



We can see that approximately 90% of the variability in the data can be explained by the top 100 principal components, and almost all can be explained using 300 PCs. To choose the number of PCs, a rule of thumb is to simply eyeball the cumulative plot, and look for a point at which the proportion of variance explained by each subsequent principal component drops off. This point is often called an elbow. For this dataset, an elbow point can be observed at roughly 100.

Recall that the raw data is 784-dimensional and are impossible to visualise. However, we can visualise the data along the first two PCs where the variance is maximised. This can be done by projecting the data onto the space spanned by the first two PCs. The projected data would then be 2-dimensional.

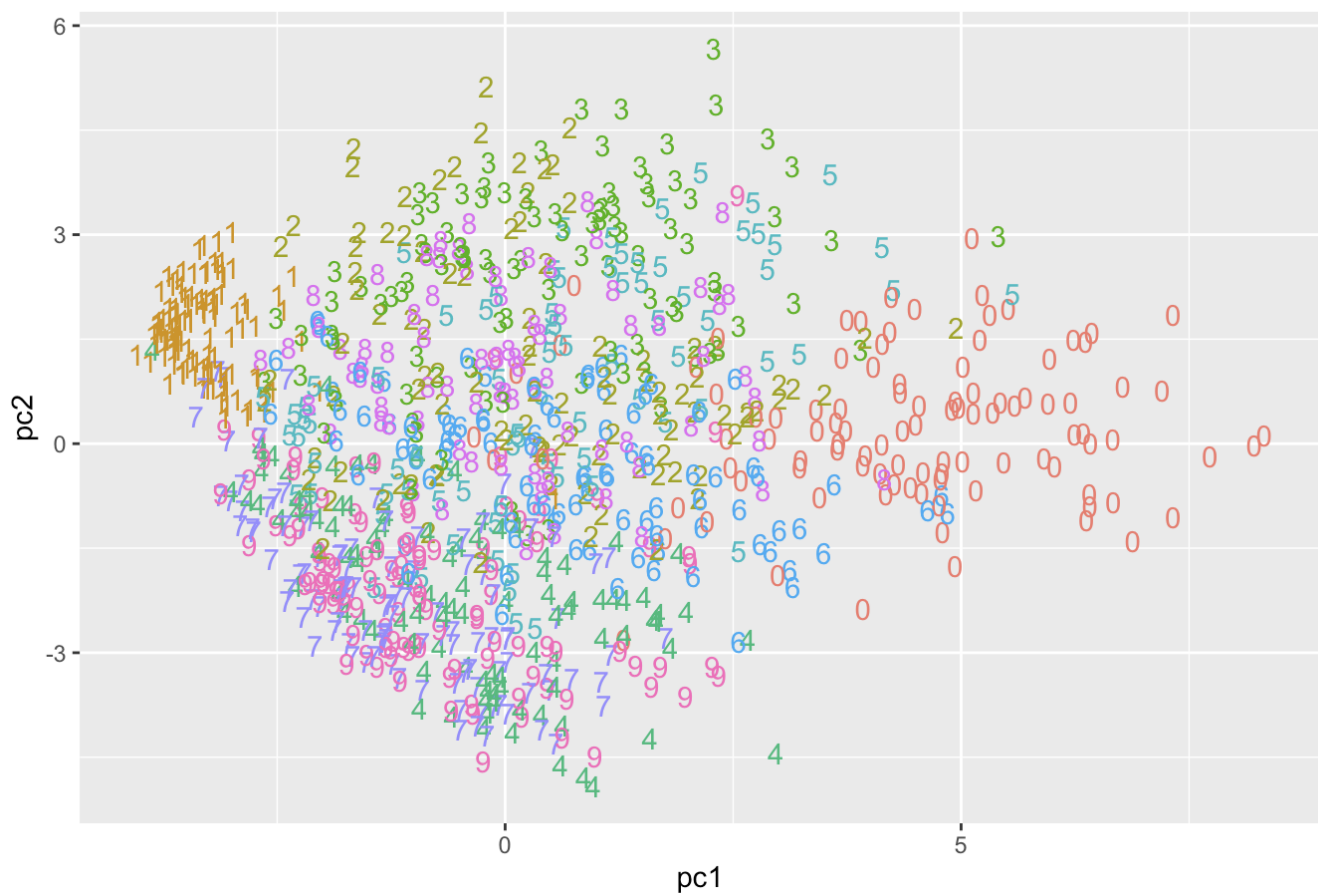
With `prcomp`, the projected data can be accessed by `mnist_pca$x`.

Exercises

4. Extract the first and the second columns of the projected data, and assign them to the variables `pc1` and `pc2`, respectively; place these in a dataframe, `pc_df`, along with the corresponding labels (i.e. from `mnist$digit`).
5. You are provided with the code to plot the projected data. Based on this plot, what would you expect if we were to run a classification model to predict the labels using only this two-dimensional projected data as features?

```
pc1 <- mnist_pca$x[, 1] # Q4
pc2 <- mnist_pca$x[, 2] # Q4
pc_df <- data.frame(pc1,pc2,digit=mnist$digit) # Q4
ggplot(pc_df, aes(x=pc1,y=pc2,colour=digit)) +
  geom_text(aes(label=digit)) +
  labs(title="PCA: projection of 1000 MNIST data observations onto first two PCs")
+
  theme(legend.position = "none")
```

PCA: projection of 1000 MNIST data observations onto first two PCs



We can see some of the digits appear to be well separated (e.g. digits 1 and 0), so if we were to run a classifier using $pc1$ and $pc2$ as features, we would expect the classification accuracy to be high between these digits. However, digits such as 2 and 3, or 4, 7 and 9, still appear to be clustered. This suggests that we would need to include more PCs as features for a high *overall* classification accuracy. Also, it makes intuitive sense that these digits are more difficult to distinguish than, say, 0 and 1.

PCA allows us to represent the data using only a few principal components. Given a number of PCs $1 \leq r \leq p$, we can also “reconstruct” the data by forming an approximation:

$$X \approx f(X; r) := \bar{X} + (X - \bar{X})U_{1:r}U_{1:r}^T$$

where $U_{1:r} = [v_1, v_2, \dots, v_r]$ is the matrix formed by column-concatenating the top r principal components v_r , and $\bar{X}$ is the matrix where the j -th column is a vector with all entries equal to the j -th column mean of X , i.e. $\bar{X} = [\bar{X}_{\cdot,1}\mathbf{1}, \bar{X}_{\cdot,2}\mathbf{1}, \dots, \bar{X}_{\cdot,p}\mathbf{1}]$ with $\mathbf{1} = (1, 1, \dots, 1)^T$. Intuitively, $f(X; r)$ is formed by firstly projecting the centralised data $X - \bar{X}$ to a r -dimensional subspace using the top r PCs with the matrix $U_{1:r}$, then projecting it back to the data space using $U_{1:r}$, and finally adding back the mean. Note that $f(X; r)$ has the same dimension as the raw data X .

With `prcomp`, the matrix of PCs $U_{1:p}$ can be accessed by `mnist_pca$rotation` which contains all p PCs as its columns, and the rotated data $(X - \bar{X})U_{1:r}$ is, as before, given by `mnist_pca$x`.

Exercise

Write a function, `reconstruct_mnist()`, that will...

6. Extract the first r PCs from `mnist_pca$rotation` and assign it to `Ur`.
7. Compute the reconstructed data by matrix-multiplying `Xcentred_Ur` by the transpose of `Ur`.

```
xbar <- colMeans(mnist_x) # compute column means for centering

reconstruct_mnist <- function(mnist_pca, r) {
  Xcentred_Ur <- mnist_pca$x[, 1:r]
  Ur <- mnist_pca$rotation[, 1:r] # Q6
  recon_centred <- Xcentred_Ur %*% t(Ur) # Q7

  n <- nrow(recon_centred)
  recon <- matrix(rep(xbar, n), byrow = T, nr = n) + recon_centred

  return(recon)
}
```

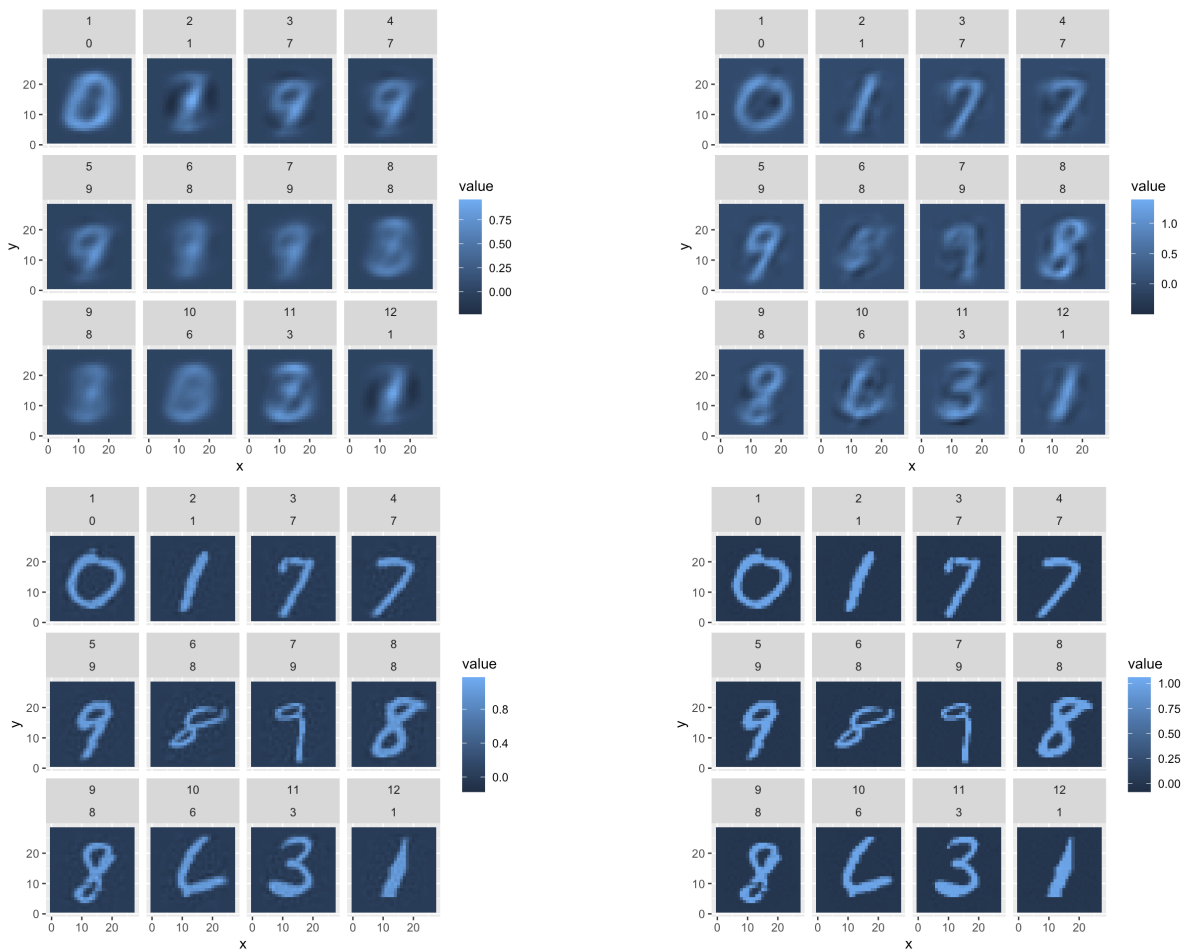
Use this function on our MNIST dataset; visualise the reconstructions using `visualise()`. Investigate how the value of r affects the quality of the reconstructed images.

```
r <- 2
recon <- reconstruct_mnist(mnist_pca, r)
visualise(recon[ind, ], label = mnist$digit[ind], nrow = 3, ncol = 4)

r <- 20
recon <- reconstruct_mnist(mnist_pca, r)
visualise(recon[ind, ], label = mnist$digit[ind], nrow = 3, ncol = 4)

r <- 200
recon <- reconstruct_mnist(mnist_pca, r)
visualise(recon[ind, ], label = mnist$digit[ind], nrow = 3, ncol = 4)

r <- 400
recon <- reconstruct_mnist(mnist_pca, r)
visualise(recon[ind, ], label = mnist$digit[ind], nrow = 3, ncol = 4)
```

8. Why does the quality of the reconstructed data increase for larger values of r ?

A larger value of r means more PCs are used, and therefore more variance is retained by the projected data. In other words, less information is lost due to projection, thus the higher reconstructed quality.

9. By making reference to the dimensionality of the rotated data $(X - \bar{X})U_{1:r}$ and the matrix of PCs $U_{1:r}$, explain why when $r \ll p$, and $r \ll n$, this decomposition requires less computational memory than storing the raw matrix X .

Storing the raw matrix X requires $O(n \times p)$ memory. The reconstruction $f(X; r)$ is computed by matrix-multiplying $(X - \bar{X})U_{1:r}$ by $U_{1:r}^T$, then adding back the matrix of means $\bar{X}$ (see `reconstruct_mnist()`), and the storage cost for these matrices are $O(n \times r)$ for $(X - \bar{X})U_{1:r}$, $O(r \times p)$ for $U_{1:r}$, and $O(p)$ for $\bar{X}$ (since each of its columns is a vector of a *single* value). Each of these cost less than $O(n \times p)$ when both $r \ll n$ and $r \ll p$.

Benefits and Limitations

Computationally, PCA costs $O(p^2 \min(n, p^2))$ operations. Besides visualisability, there are several other advantages of performing PCA when working with high dimensional data:

- **Correlation removal:** PCA removes correlation between features as the resulting PCs are orthogonal (i.e. uncorrelated). This allows us to sidestep issues such as multi-collinearity of the data, and can boost the performance of some machine learning models.
- **Reduced overfitting:** Since PCA allows us to pick a few directions that can explain most of the variability of the data, it helps reduce overfitting of models by removing redundant features.

Some limitations of PCA are:

- **Linear relationship:** PCA is based on the variance/covariance of the data, which describe the linear

relationships between variables. PCA is not capable of interpreting nonlinear (e.g. polynomial) relationships between the features.

- Loss of interpretability: The PCs are arbitrary unit vectors that are linear combinations of the original features, and can therefore be extremely difficult to interpret.

Multidimensional Scaling

Multidimensional scaling (MDS) is a means of accurately retaining the *dissimilarities* between observations in a dataset, whilst representing those observations in a low-dimensional space. An MDS algorithm takes in a matrix of pairwise dissimilarity measures, and returns a set of points in the lower-dimensional 'MDS space' whose (typically Euclidean) relative distances accurately represent these dissimilarities.

Since the MDS representation of a dataset is specified by these relative distances in low-dimensional space, we can subject this representation to similarity transformations (i.e. geometric transformations that preserve the relative distances between objects: translations, rotations, reflections and scalings), without impacting its representation of the original dataset.

There are two common variants of MDS: ratio/metric MDS and ordinal/non-metric MDS. As you will have seen in this week's reading material, in ratio MDS, the reconstruction points are chosen such that their relative distances are approximately proportional to the corresponding dissimilarity measures. In contrast, with ordinal MDS, the relative distances between the points in MDS space need only satisfy the order relations induced by the pairwise dissimilarities.

Metric and non-metric MDS can be unified in the following framework. Given a data matrix $X \in \mathbb{R}^{n \times p}$ with corresponding dissimilarities $d_{ij} = d(x_i, x_j) \in \mathbb{R}^{n \times n}$, the goal of MDS is to find a matrix $Y \in \mathbb{R}^{n \times k}$, with $k \ll p$ (and often using $k = 2$ or $k = 3$) such that the pairwise (Euclidean) distance between rows i and j , $\delta_{ij} = \|y_i - y_j\|_2$, is as close as possible to some function of the corresponding dissimilarity. In other words, we minimise the *squared representation error* for each dissimilarity:

$$e_{ij}^2 = \{f(d_{ij}) - \delta_{ij}\}^2,$$

where $f: \mathbb{R} \mapsto \mathbb{R}$ is the *representation function*. Aggregating the squared representation errors for all discrepancies yields the *raw Stress*; the goal of MDS is to minimise this aggregated term. Note, however, that the value of the raw Stress (squared) error will depend on the scale of the representation distances δ_{ij} . In practice, we therefore typically seek to minimise the (normed) Stress

$$\sigma_1^2 = \frac{\sum_{i,j} \{f(d_{ij}) - \delta_{ij}\}^2}{\sum_{i,j} \delta_{ij}^2}.$$

Furthermore, since the goal is minimisation of σ_1^2 , MDS algorithms typically target minimisation of σ_1 (i.e. the positive square root), in order to better discriminate between small values of the targeted loss.

The choice of representation function determines the type of MDS that is performed, and should largely be determined by the level of measurement (i.e. the NOIR classification) of the dissimilarity matrix. If the dissimilarities are treated as ratio quantities, then the representation function should be linear, with zero intercept: $f(d_{ij}) = \alpha d_{ij}$, leading to ratio MDS. (Similarly, *interval* MDS is obtained by using a linear representation function with nonzero intercept, and is suitable if the dissimilarities are interval quantities, but not ratio quantities.) If the dissimilarities are ordinal quantities, then f can be any monotonic (non-decreasing) function, and this results in ordinal MDS.

In the specific case where the original dissimilarities d_{ij} are, themselves, Euclidean distances, the solution to the ratio MDS problem can be found through applying a technique known as *classical scaling* (also,

Torgerson or *Torgerson-Gower scaling*, due to its discovery by Torgerson (1952,1958) and Gower (1966)). In R, the functions for classical scaling and ordinal MDS are `cmdscale` (which can be found in the `stats` package) and `isoMDS` (from the `MASS` package), respectively. Note that both the `stats` and `MASS` packages are automatically included with most installations of R. You can uncomment the following lines of code to access the R documentation for these functions.

```
# ?cmdscale # classical scaling
# ?MASS::isoMDS # ordinal MDS
```

We apply classical scaling to the MNIST dataset. The `k` option in `cmdscale` controls the dimensionality of the space in which the data are to be represented in. Execution of the following code chunk may take a few seconds.

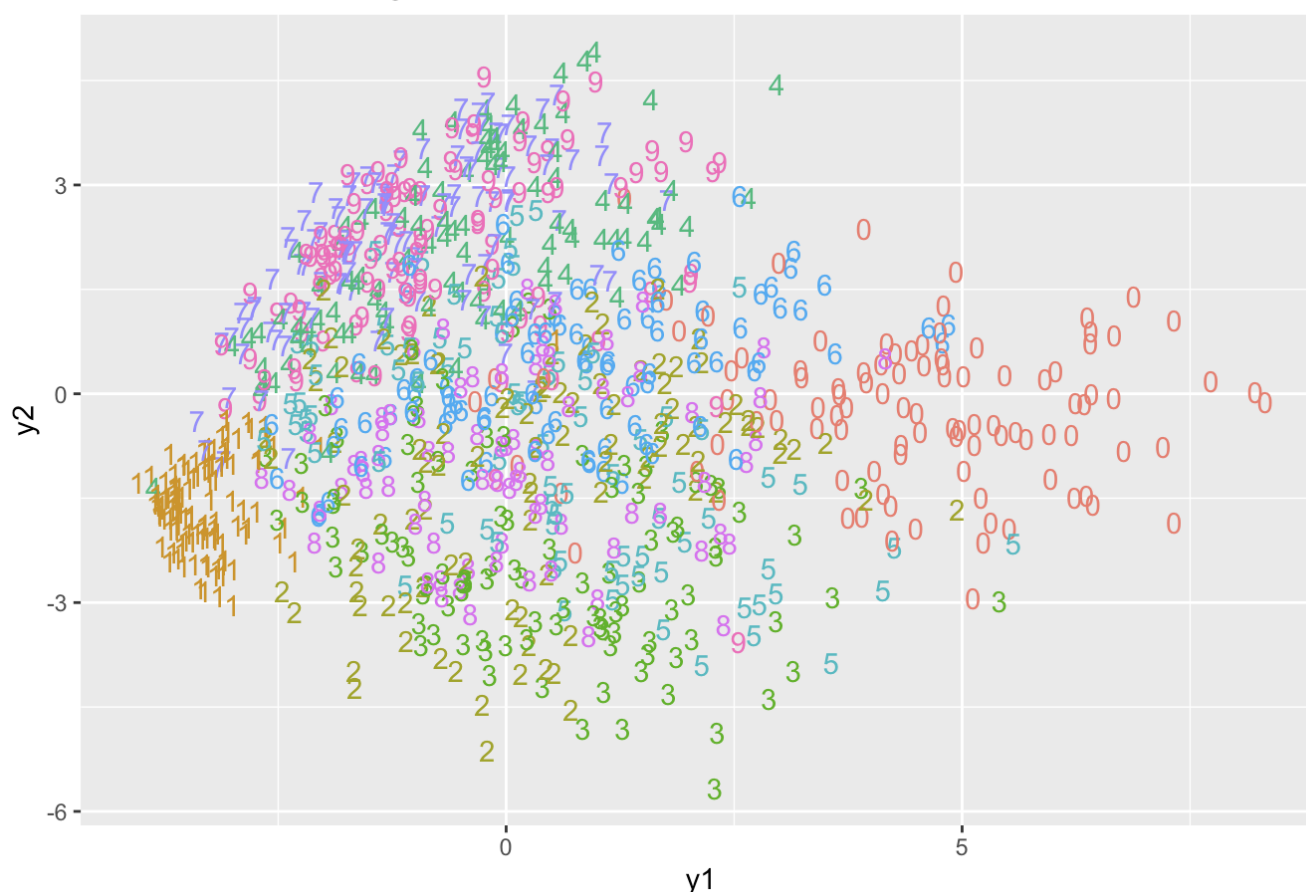
```
mnist_dist <- dist(mnist_x) # euclidean distances between the rows
mds <- cmdscale(mnist_dist, eig = TRUE, k = 2) # k is the dim of the reduced space
summary(mds) # view results
```

```
##           Length Class  Mode
## points 2000    -none- numeric
## eig    1000    -none- numeric
## x        0     -none-  NULL
## ac        1     -none- numeric
## GOF        2     -none- numeric
```

```
mds_df <- data.frame(y1=mds$points[,1],
                    y2=mds$points[,2],
                    digit=mnist$digit)

ggplot(mds_df, aes(x=y1,y=y2,colour=digit)) +
  geom_text(aes(label=digit)) +
  labs(title="MDS: 2D embedding of 1000 MNIST data observations") +
  theme(legend.position = "none")
```

MDS: 2D embedding of 1000 MNIST data observations



We can see that, by using just two components, some digits already appear to be clustered, such as the digits 1 and 0. If we run a classification model on the data for the digit 1, say, against the others, then we would expect it to have a reasonably decent accuracy, despite the fact that we have reduced the dimensionality from 784 to merely 2!

Let us repeat this analysis with ordinal MDS, and compare the results with those of classical scaling.

Exercise:

10. Complete the code below to implement ordinal MDS by supplying the pairwise Euclidean distance matrix into `MASS::isoMDS`. Set the dimension of the solution to 2.
11. Compare with the results of classical scaling. Do you observe any differences?

```
ordinal_mds <- MASS::isoMDS(mnist_dist, k = 2) # Q10
```

```
## initial value 36.980310
## iter 5 value 31.555757
## iter 10 value 29.452280
## final value 29.374198
## converged
```

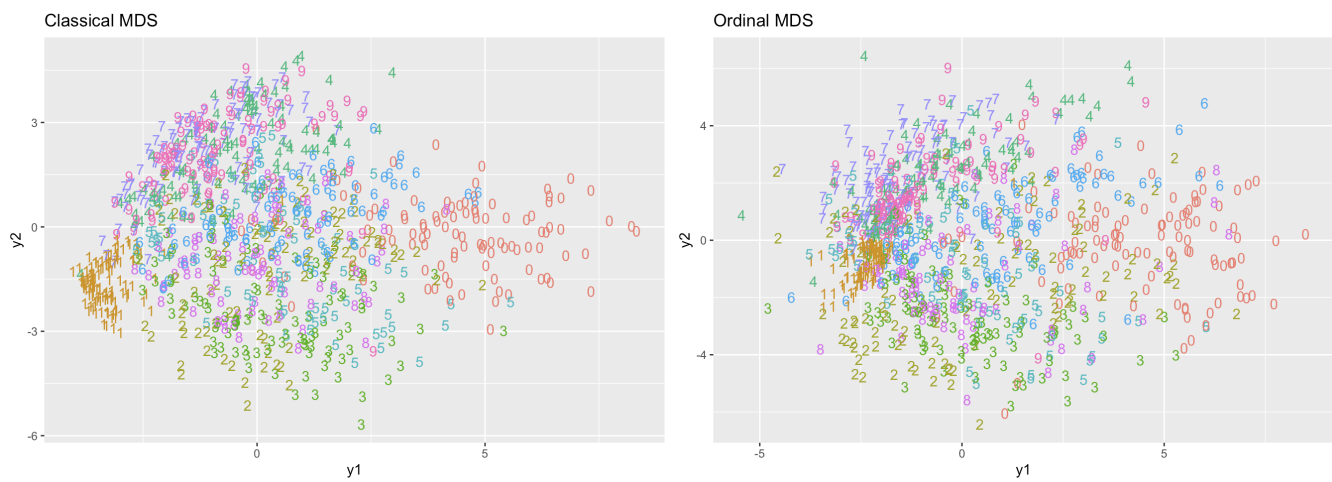
```
summary(ordinal_mds) # view results
```

```
##      Length Class Mode
## points 2000  -none- numeric
## stress   1   -none- numeric
```

```
ordinalmds_df <- data.frame(y1=ordinal_mds$points[,1],
                           y2=ordinal_mds$points[,2],
                           digit=mnist$digit)

ggplot(mds_df, aes(x=y1,y=y2,colour=digit)) +
  geom_text(aes(label=digit)) +
  labs(title="Classical MDS") +
  theme(legend.position = "none")

ggplot(ordinalmds_df, aes(x=y1,y=y2,colour=digit)) +
  geom_text(aes(label=digit)) +
  labs(title="Ordinal MDS") +
  theme(legend.position = "none")
```



The clustering patterns are largely the same for classical and ordinal MDS in this case, suggesting that both methods can lead to similar set of reconstructed points. (Care would need to be taken if they produced completely different results - we would need to think carefully about which approach is more suitable for the dataset in question!)

Benefits and Limitations

The computational time for MDS is in general $O(n^3)$ (excluding the cost for computing the dissimilarities) due to the need to perform a QR factorisation on the matrix of pairwise distances, which is of order $n \times n$.

This method has several advantages:

- **Simplicity:** It provides a simple and intuitive way to understand the similarity of data points based on their relative distances.
- **Applicability:** It is highly applicable to data of various types due to the simple assumptions it is making. In particular, ordinal MDS only assumes the pairwise dissimilarities are interpretable in an ordinal sense.

The main limitation of MDS is its lack of scalability with respect to the number of observations. The cubic computational cost in n means that applying MDS to large datasets can be infeasible, making it unsuitable for applications such as genome array data with a large number of genes.

t-distributed Stochastic Neighbour

Embedding

t-Distributed Stochastic Neighbour Embedding (t-SNE) is another dimensionality reduction method that aims to preserve the dissimilarity between observations.

As above, the overarching goal here is to map each p -dimensional data point $x_i \in \mathbb{R}^p$ to a point in some low-dimensional space, say $y_i \in \mathbb{R}^2$. With t-SNE, we allow the low-dimensional representation to be *random*, such that the probability of two embedded points being close to one another reflects the similarity of the two corresponding high-dimensional data observations.

More specifically, let us define the similarity of observations $x_i, x_j \in \mathbb{R}^p$ to be $p_{ij} = (p_{i|j} + p_{j|i})/(2n)$, where

$$p_{j|i} := \frac{\exp\{-d_{ij}^2/(2\sigma_i^2)\}}{\sum_{k \neq i} \exp\{-d_{ik}^2/(2\sigma_i^2)\}}$$

is the (asymmetric) similarity of observation x_j to x_i . Here, $d_{ij} = \|x_i - x_j\|_2$, as before. This defines the (asymmetric) similarity of observation x_j to observation x_i using a Gaussian distribution, centred around x_i and with a given variance σ_i^2 . Note that $p_{i|j}$ is an asymmetric similarity measure due to the fact that the bandwidth / standard deviation parameters, σ_i , vary with i . In implementation, the values of these parameters are chosen so that the *perplexity* of each of the corresponding (conditional) Gaussian distributions is approximately equal to a pre-specified value.

Let us also define the similarity between points y_i and y_j in the reduced-dimension embedding, using the following similarity measure:

$$q_{ij} := \frac{(1 + \|y_i - y_j\|_2^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|_2^2)^{-1}}.$$

Here, we define the similarity between embedded points using a *t-distribution* with 1 degree-of-freedom (hence the letter “t” in the name).

Note that both $\{p_{ij} ; i, j = 1, \dots, n\}$ and $\{q_{i,j} ; i, j = 1, \dots, n, i \neq j\}$ define (discrete) probability distributions. We can therefore optimise the low-dimensional representation of the data by minimising the discrepancy between these two distributions. Indeed, t-SNE does so by iteratively minimising the KL-divergence of the distribution $\{p_{ij} ; i, j = 1, \dots, n\}$ from the distribution $\{q_{i,j} ; i, j = 1, \dots, n, i \neq j\}$.

In R, we can implement t-SNE using the function `Rtsne()` from the `Rtsne` package! You can uncomment the following lines of code to access the R documentation for this functions.

```
# ?Rtsne::Rtsne
```

The `dim` option controls the dimensionality of the reduced space, the perplexity of each of the conditional Gaussian distributions is controlled by `perplexity`, and `max_iter` is the maximal number of iteration for finding the optimal solution.

See this website (<https://distill.pub/2016/misread-tsne/>) for an interactive interface where you can play around with different values of perplexity. An animation of the convergence of t-SNE on MNIST with the number of iteration can be found on this blog (<https://www.oreilly.com/content/an-illustrated-introduction-to-the-t-sne-algorithm/#:~:text=We%20create%20an%20animation%20using%20MoviePy.>).

Exercises

12. In the following code chunk, use `Rtsne()` find a two-dimensional embedding of our chosen subset of the MNIST data. Use a perplexity parameter of 30.
13. By plotting the resulting embedding, compare your results with those found above using PCA and MDS.

Below is an implementation of t-SNE for our chosen subset of the MNIST data. Note that the t-SNE algorithm is a *stochastic* one, so each run will give different results in general; this can be seen by modifying the random seed in the below chunk (which we have only set to allow reproducibility).

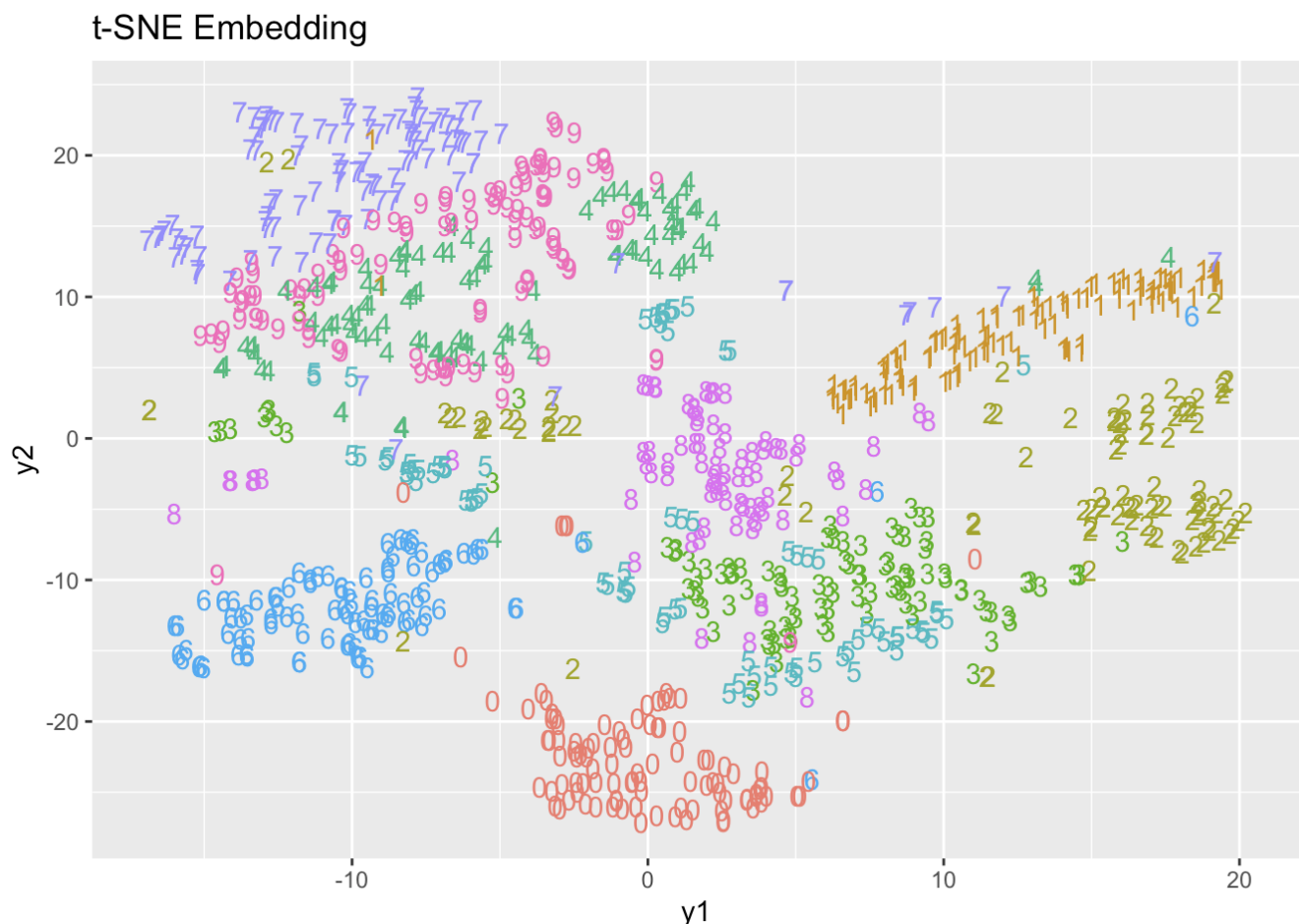
```
set.seed(2022)

# run tsne; verbose = TRUE enables progress messages
tsne <- Rtsne(mnist_x, dims = 2, perplexity = 30, verbose = TRUE, max_iter = 500)
```

```
## Performing PCA
## Read the 1000 x 50 data matrix successfully!
## Using no_dims = 2, perplexity = 30.000000, and theta = 0.500000
## Computing input similarities...
## Building tree...
## Done in 0.20 seconds (sparsity = 0.123364)!
## Learning embedding...
## Iteration 50: error is 70.109897 (50 iterations in 0.20 seconds)
## Iteration 100: error is 69.539812 (50 iterations in 0.19 seconds)
## Iteration 150: error is 69.072397 (50 iterations in 0.17 seconds)
## Iteration 200: error is 69.066407 (50 iterations in 0.17 seconds)
## Iteration 250: error is 69.067485 (50 iterations in 0.18 seconds)
## Iteration 300: error is 1.290898 (50 iterations in 0.16 seconds)
## Iteration 350: error is 1.112841 (50 iterations in 0.15 seconds)
## Iteration 400: error is 1.066520 (50 iterations in 0.16 seconds)
## Iteration 450: error is 1.051030 (50 iterations in 0.16 seconds)
## Iteration 500: error is 1.039070 (50 iterations in 0.15 seconds)
## Fitting performed in 1.68 seconds.
```

```
tsne_df <- data.frame(y1=tsne$Y[,1],
                     y2=tsne$Y[,2],
                     digit=mnist$digit)

ggplot(tsne_df, aes(x=y1,y=y2,colour=digit)) +
  geom_text(aes(label=digit)) +
  labs(title="t-SNE Embedding") +
  theme(legend.position = "none")
```



Looking at the plots of two-dimensional projections produced by each of the three methods, we see that tSNE yields the most well-separated clusters. This separability would mean that a (non-linear) classifier using only these two dimensions would be able to give reasonable accuracy when classifying all 10 digits. PCA and MDS performed similarly but both are clearly worse than tSNE, so a classifier based on the two components produced by PCA or MDS would perform well on simple digit pairs such as 0 and 1, and struggle on more difficult ones such as 4, 7 and 9.

Benefits and Limitations

The t-SNE algorithm has a computational complexity of $O(n^2)$ *per iteration*, since *pairwise* dissimilarities are required. Its main advantages include

- **Non-linear relationship:** Unlike PCA which only considers the linear relationships, t-SNE is able to capture the non-linear relationship between data points, because it tries to learn a low-dimensional representation that respects some dissimilarity measures that are non-linear to the input.
- **Preservation of local and global structure:** t-SNE is capable of preserving local and global structure of the data, so that points that are close together in the original dimension tend to be close also in the low dimension.

Some of its disadvantages are:

- **Computational complexity:** It is computationally expensive as at each iteration the pairwise distances need to be calculated. This is particularly the case compared with PCA, which is a cheaper method.
- **Hyperparameter tuning:** The perplexity parameter requires manual tuning in order to achieve a good performance; that is, it is a *hyperparameter*. One way to tune this hyperparameter is by using cross-validation.

References

1. Torgerson, W.S., 1952. Multidimensional scaling: I. Theory and method. *Psychometrika*, 17(4), pp.401-419.
2. Torgerson, W. S., 1958. *Theory and methods of scaling*. Wiley.
3. Gower, J.C., 1966. Some distance properties of latent root and vector methods used in multivariate analysis. *Biometrika*, 53(3), pp325-338.
4. Rossant, C., 2022. *An illustrated introduction to the t-SNE algorithm*. [online] O'Reilly Media. Link (<https://www.oreilly.com/content/an-illustrated-introduction-to-the-t-sne-algorithm/>) [Accessed 1 March 2022].
5. Wattenberg, M., Viégas, F., and Johnson, I., 2016. *How to Use t-SNE Effectively*. Distill. Link (<https://distill.pub/2016/misread-tsne/>)